

More Kotlin fundamentals

The Kotlin behind every list in Compose: smarter classes, collections, and higher-order functions.

Today

3 codelabs + practice

Runs in

Kotlin Playground

Sets you up for

Lab 4: Scrollable lists

Based on the content of Android Basics with Compose, Unit 3, Pathway 1: More Kotlin fundamentals
developer.android.com/courses/pathways/android-basics-compose-unit-3-pathway-1

From last week to this week

Same language, bigger toolbox

LAST WEEK · UNIT 2

Conditionals

`if / else, when`

Nullability

`String?, ?., ?:, !!`

Classes and objects

`class, constructor, inheritance`

Function types and lambdas

`val f: (Int) → Int = { it * 2 }`

THIS WEEK · UNIT 3

Generics

`class Question<T>`

Special classes and extensions

`enum, data, object, let, apply`

Collections

`Array, List, Set, Map`

Higher-order functions

`forEach, map, filter, groupBy, fold`

Today in three parts

Each part ends with a quick check

PART 1

Generics, objects, and extensions

Running example

A quiz app

`Question<T>`, `enum`, `data`
`object`, `let`, `apply`

PART 2

Collections

Running example

The solar system

`arrayOf`, `listOf`
`setOf`, `mapOf`

PART 3

Higher-order functions

Running example

A cookie menu

`forEach`, `map`, `filter`
`groupBy`, `fold`, `sortedBy`

WHERE IT ALL MEETS

Lab 4: Scrollable lists

`data class`

describes one item

`List`

holds all the items

`lambda`

draws each item

Generics, objects, and extensions

Kotlin features that help you structure larger apps. We build the data model of a quiz app step by step.

Reuse

`Question<T>`

Model

`enum / data / object`

Extend

`extensions / let / apply`

Three classes, one difference

FILL IN THE BLANK

```
class FillInTheBlankQuestion(  
    val questionText: String,  
    val answer: String,  
    val difficulty: String  
)
```

TRUE OR FALSE

```
class TrueOrFalseQuestion(  
    val questionText: String,  
    val answer: Boolean,  
    val difficulty: String  
)
```

MATH PROBLEM

```
class NumericQuestion(  
    val questionText: String,  
    val answer: Int,  
    val difficulty: String  
)
```

All three classes have the exact same properties. The only difference is the data type of **answer**.

Inheritance does not fix it: every new question type would still need its own answer property.

What is a generic type?

```
class class name < generic data type >(  
    val property name : generic data type  
)
```

The placeholder is declared in angle brackets after the class name, then used like any real data type inside the class.

A placeholder for a type

Generics let a class use an unknown data type for its properties and methods.

Decided when you create the object

The real type (String, Int, Boolean...) is given when the class is instantiated.

Usually named T

Short for type. It is a convention, not a rule: any name works.

One class for every question type

KOTLIN

```
class Question<T>(  
    val questionText: String,  
    val answer: T,  
    val difficulty: String  
)
```

val instance name =
class name < generic data type > (parameters)

The value you pass for the generic property must match the type in the angle brackets.

THREE INSTANCES, ONE CLASS

```
val question1 = Question<String>("Quoth the raven ___", "nevermore", "medium")  
val question2 = Question<Boolean>("The sky is green. True or false", false, "easy")  
val question3 = Question<Int>("How many days are there between full moons?", 28, "hard")
```

The problem with strings

KOTLIN

```
Question<String>("Quoth the raven ___", "nevermore", "medium")
Question<String>("Quoth the raven ___", "nevermore", "meduim")    // typo, still compiles
Question<String>("Quoth the raven ___", "nevermore", "impossible") // not a valid level, still compiles
```

Typos become bugs

Mistype one of the three strings and nothing warns you.

Renaming is painful

If "medium" becomes "average", every usage must be updated.

Any string is accepted

Nothing stops a value that is not one of the three valid ones.

Harder to maintain

Adding more difficulty levels makes all of this worse.

Enum class: a limited set of values

```
enum class enum name {  
    Case 1 , Case 2 , Case 3  
}
```

Refer to a constant with the dot operator

```
enum name . case name
```

KOTLIN

```
enum class Difficulty {  
    EASY, MEDIUM, HARD  
}  
  
class Question<T>(  
    val questionText: String,  
    val answer: T,  
    val difficulty: Difficulty  
)
```

Each value is an **enum constant**, written in capitals by convention. A typo such as Difficulty.MEDUIM no longer compiles.

Data class: a class that holds data

KOTLIN

```
data class Question<T>(  
    val questionText: String,  
    val answer: T,  
    val difficulty: Difficulty  
)  
  
println(question1.toString())
```

OUTPUT WITHOUT DATA

Question@37f8bb67

OUTPUT WITH DATA

Question(questionText=Quoth the raven ____,
answer=nevermore, difficulty=MEDIUM)

Question only holds data, it has no methods that perform an action. Add the keyword **data** before class and Kotlin implements `toString()` and other methods from the properties.

What a data class gives you

<code>equals()</code>	Two objects with the same property values are considered equal.
<code>hashCode()</code>	Used when working with certain collection types.
<code>toString()</code>	Readable text built from the class's properties.
<code>componentN()</code>	<code>component1()</code> , <code>component2()</code> , and so on: one per property.
<code>copy()</code>	A new object with the same values, optionally changing some.

Rules: at least one constructor parameter, all marked `val` or `var`. A data class cannot be abstract, open, sealed, or inner.

Singleton: a class with one instance

```
object object name {  
    class body  
}
```

Use the object keyword instead of class. There is no constructor: properties get their initial values inside the braces.

Player stats

The current user's stats in a mobile game.

A single hardware device

For example, sending audio through a speaker.

A remote data source

One object to access a database such as Firebase.

Authentication

Only one user should be logged in at a time.

Define and access a singleton object

KOTLIN

```
object StudentProgress {  
    var total: Int = 10  
    var answered: Int = 3  
}  
  
fun main() {  
    println("${StudentProgress.answered} of ${StudentProgress.total} answered.")  
}
```

Output: 3 of 10 answered.

object name

.

property name

You cannot create an instance yourself. Use the name of the object, a dot, then the property.

One shared state

Every part of the program reads and updates the same total and answered.

Companion object: inside a class

KOTLIN

```
class Quiz {  
    // question1, question2, question3  
  
    companion object StudentProgress {  
        var total: Int = 10  
        var answered: Int = 3  
    }  
}  
  
fun main() {  
    println("${Quiz.answered} of ${Quiz.total} answered.")  
}
```

Output: 3 of 10 answered.

One extra keyword

Put companion before object, inside the class.

Access through the class name

Quiz.answered and Quiz.total, even though they are declared in StudentProgress.

When to use it

When the object's properties and methods belong to that class.

Where do .dp and .sp come from?

COMPOSE

```
Text(  
    text = "Hello",  
    fontSize = 24.sp,  
    modifier = Modifier.padding(16.dp)  
)
```

Numbers with UI properties?

In Compose, numbers appear to have properties like dp and sp. Kotlin's designers did not plan for Compose.

Kotlin lets you extend existing types

Any developer can add properties and methods to a data type, used with dot syntax as if they were part of it.

No access to the source needed

The Compose library added dp and sp without changing Kotlin's number types.

Add an extension property

val type name . property name : data type

property getter

Add the type name and a dot before the property name.

KOTLIN

```
val Quiz.StudentProgress.progressText: String  
    get() = "${answered} of ${total} answered"
```

```
fun main() {  
    println(Quiz.progressText)  
}
```

Output: 3 of 10 answered

Extension properties can't store data, so they must be **get-only**.

Add an extension function

```
fun type name . function name ( parameters ) : return type { function body }
```

KOTLIN

```
fun Quiz.StudentProgress.printProgressBar() {  
    repeat(Quiz.answered) { print("█") }  
    repeat(Quiz.total - Quiz.answered) { print("░") }  
    println()  
    println(Quiz.progressText)  
}
```

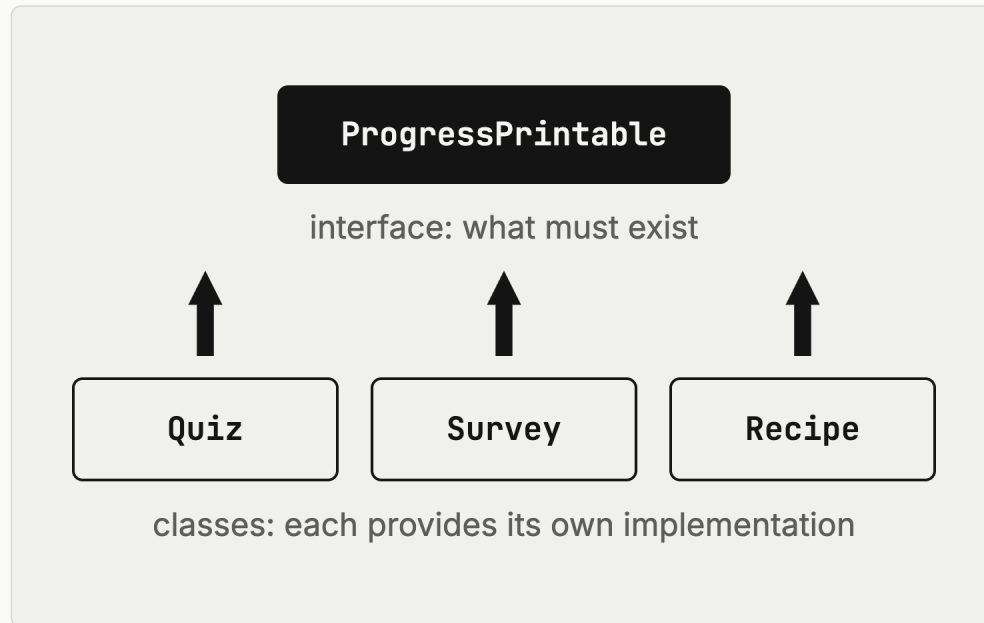
CALL IT IN MAIN()

```
Quiz.printProgressBar()
```

OUTPUT

```
████░░░░░░  
3 of 10 answered
```

Interfaces: a contract for classes



KOTLIN

```
interface ProgressPrintable {  
    val progressText: String  
    fun printProgressBar()  
}  
  
class Quiz : ProgressPrintable {  
    ...  
}
```

An interface is a **contract**. A class that extends it must implement every property and method it lists.

The progress bar as an interface

KOTLIN

```
class Quiz : ProgressPrintable {  
    override val progressText: String  
        get() = "${answered} of ${total} answered"  
  
    override fun printProgressBar() {  
        repeat(Quiz.answered) { print("█") }  
        repeat(Quiz.total - Quiz.answered) { print("░") }  
        println()  
        println(progressText)  
    }  
}
```

IN MAIN()

```
Quiz().printProgressBar()
```

Output:



3 of 10 answered

What changed

Both members are marked override and now live inside Quiz, so the two extensions are deleted. Create a Quiz first, then call the method.

Scope functions: let

BEFORE

```
fun printQuiz() {  
    println(question1.questionText)  
    println(question1.answer)  
    println(question1.difficulty)  
    println()  
    // same for question2 and question3  
}
```

WITH LET

```
fun printQuiz() {  
    question1.let {  
        println(it.questionText)  
        println(it.answer)  
        println(it.difficulty)  
    }  
    println()  
    // same for question2 and question3  
}
```

Scope functions give access to an object's properties and methods without repeating its name. With `let`, the object is called **it** inside the lambda.

Scope functions: apply

BEFORE

```
val quiz = Quiz()  
quiz.printQuiz()
```

WITH APPLY

```
Quiz().apply {  
    printQuiz()  
}
```

OUTPUT

```
Quoth the raven ___  
nevermore  
MEDIUM
```

```
The sky is green. True or false  
false  
EASY
```

```
How many days are there between full moons?  
28  
HARD
```

apply runs the lambda on the object and returns that same object, so you can call its methods without a variable.

Summary

01

Generics pass a data type to a class as a parameter: `Question<T>`.

02

Enum classes define a limited set of possible values.

03

Data classes generate `toString()`, `equals()`, `copy()` and more for you.

04

An object is a singleton: one instance. A companion object lives inside a class.

05

Extensions add get-only properties and new methods to existing classes.

06

Interfaces set a contract. Scope functions like `let` and `apply` shorten access to an object.

Which line does not compile?

```
class Question<T>(val text: String, val answer: T)

val a = Question<String>("Capital of Canada?", "Ottawa")
val b = Question<Int>("2 + 2 = ?", "4")
val c = Question<Boolean>("The sky is green.", false)
```

A Line a

B Line b

C Line c

D They all compile

Line b: the value must match T

```
class Question<T>(val text: String, val answer: T)

val a = Question<String>("Capital of Canada?", "Ottawa")
val b = Question<Int>("2 + 2 = ?", 4)
val c = Question<Boolean>("The sky is green.", false)
```

Why

Question<Int> means answer has type Int. The value **"4"** is in quotes, so it is a String and the types do not match.

The fix

Pass **4** without quotes, as shown here, or declare it as **Question<String>**.

What happens here?

```
enum class Difficulty {  
    EASY, MEDIUM, HARD  
}  
  
val level = Difficulty.MEDUIM
```

- A level holds MEDUIM
- B level is null
- C Compile error
- D A new constant is created

Compile error: the typo is caught

```
enum class Difficulty {  
    EASY, MEDIUM, HARD  
}  
  
val level = Difficulty.MEDUIM  
// error: unresolved reference MEDUIM
```

Why

The enum lists only EASY, MEDIUM and HARD. MEDUIM is a typo, so Kotlin refuses to compile the last line.

The fix

Write **Difficulty.MEDIUM**. With a String, the typo "meduim" would have compiled and become a hidden bug.

What does this print?

```
class Point(val x: Int, val y: Int)

fun main() {
    val p = Point(1, 2)
    println(p)
}
```

A Point(x=1, y=2)

B Something like Point@37f8bb67

C 1, 2

D Compile error

Point@...: it is not a data class

```
data class Point(val x: Int, val y: Int)

fun main() {
    val p = Point(1, 2)
    println(p)
}
```

Why

A regular class has no generated `toString()`, so `println` shows only the class name and an identifier.

The fix

Add **data** before class, as shown here. The same code then prints `Point(x=1, y=2)`.

How do you read the progress text?

```
class Quiz {  
    companion object StudentProgress {  
        var total: Int = 10  
        var answered: Int = 3  
    }  
}  
  
val Quiz.StudentProgress.progressText: String  
    get() = "${answered} of ${total} answered"
```

A Quiz().progressText

B Quiz.progressText

C StudentProgress().progressText

D progressText

Use the class name: Quiz.progressText

```
class Quiz {  
    companion object StudentProgress {  
        var total: Int = 10  
        var answered: Int = 3  
    }  
}  
  
val Quiz.StudentProgress.progressText: String  
    get() = "${answered} of ${total} answered"
```

Why

StudentProgress is the companion object of Quiz, so its members, including extensions added to it, are reached through the class name.

Why not the others

An object cannot be instantiated, and progressText belongs to the companion, not to an instance created with **Quiz()**.

Use collections in Kotlin

Collections store multiple values in an organized way. We explore them with the planets of the solar system.

Ordered

Array / List

Unique

Set

Key to value

Map

What is an array?

SINGLE VALUE

8

ARRAY

8	6	7	5	3	0	9
0	1	2	3	4	5	6

An index is the distance from the first element, so counting starts at 0. This is called zero-indexing.

Elements of one type

An array is a sequence of values, called elements or items, that all have the same data type.

Ordered, accessed by index

The first element is at index 0, the second at index 1, and so on.

Fast access, fixed size

Reading any index is equally fast, but you cannot add elements beyond the array's size.

Create an array with arrayOf()

```
val variable name = arrayOf<data type>( element1 , element2 , ... )
```

KOTLIN

```
val rockPlanets = arrayOf<String>("Mercury", "Venus", "Earth", "Mars")  
val gasPlanets = arrayOf("Jupiter", "Saturn", "Uranus", "Neptune")  
val solarSystem = rockPlanets + gasPlanets
```

Any number of elements

arrayOf() takes as many arguments as you give it.

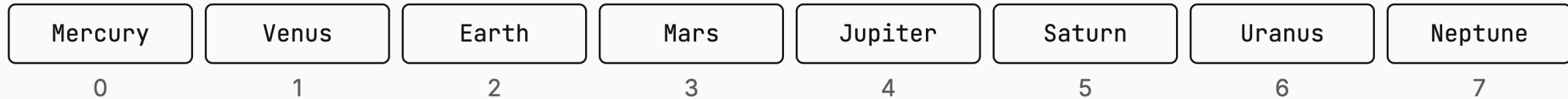
Type inference

The type in angle brackets can be omitted: Kotlin works it out.

+ joins two arrays

The result is a new array with the elements of both.

Access and change elements



KOTLIN

```
println(solarSystem[0])           // Mercury
println(solarSystem[7])           // Neptune

solarSystem[3] = "Little Earth"
println(solarSystem[3])           // Little Earth
```

READ AN ELEMENT

array name [**index**]

CHANGE AN ELEMENT

array name [**index**] = **new value**

This is called subscript syntax.

An array has a fixed size

TRY TO ADD A NINTH PLANET

```
solarSystem[8] = "Pluto"
```

RESULT

```
Exception in thread "main"  
java.lang.ArrayIndexOutOfBoundsException: Index  
8 out of bounds for length 8
```

THE ONLY WAY: A NEW, BIGGER ARRAY

```
val newSolarSystem = arrayOf(  
    "Mercury", "Venus", "Earth", "Mars", "Jupiter",  
    "Saturn", "Uranus", "Neptune", "Pluto"  
)  
println(newSolarSystem[8])
```

Output: Pluto

Adding and removing elements, keeping values unique, or mapping one object to another gets messy with arrays. That is why Kotlin has **List**, **Set** and **Map**.

List: ordered and resizable

OLD ARRAY, FULL



NEW, BIGGER ARRAY



Behind the scenes a list is an array that gets swapped for a bigger one when needed.

Resizable

When the array is full and you add an element, it is copied to a new bigger array. The list does this for you.

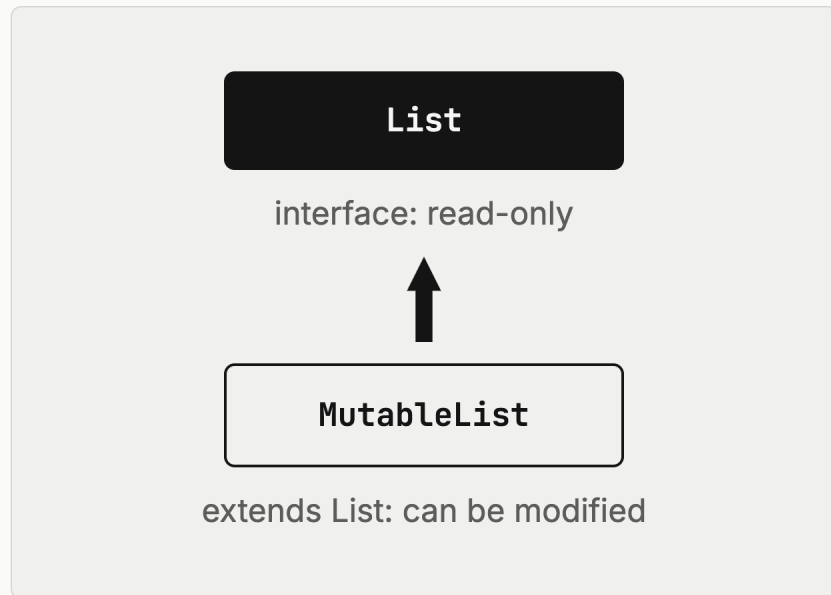
Insert anywhere

A new element can go at a specific index. Elements with higher indexes are moved.

Usually just as fast

In most cases adding an element takes the same time, however many elements the list has.

List and MutableList



List

Properties and methods of a read-only ordered collection.

`listOf()` · `size` · `get()` · `indexOf()` · `contains()`

MutableList

Extends List with methods to modify the list.

`mutableListOf()` · `add()` · `remove()` · `removeAt()`

These are **interfaces**, like in Part 1: they say what a list must offer. The implementation you will almost always use is the resizable array.

Create a list and read from it

KOTLIN

```
val solarSystem = listOf("Mercury", "Venus", "Earth", "Mars",  
    "Jupiter", "Saturn", "Uranus", "Neptune")  
  
println(solarSystem.size)           // 8  
println(solarSystem[2])             // Earth  
println(solarSystem.get(3))         // Mars  
println(solarSystem.indexOf("Earth")) // 2  
println(solarSystem.indexOf("Pluto")) // -1
```

size

A property holding the number of elements in the list.

[] and get()

Both take an index and return the element at that index. Lists are zero-indexed.

indexOf()

Returns the index of the first occurrence, or -1 if the element is not in the list.

Loop through a list with for

```
for ( element name in collection name ) {  
    body  
}
```

The body runs once for each element. Each run is called an iteration.

The variable is not declared with `val` or `var`. A list called `planets` usually gets a variable called `planet`.

KOTLIN

```
for (planet in solarSystem) {  
    println(planet)  
}
```

Output:

```
Mercury  
Venus  
Earth  
Mars  
Jupiter  
Saturn  
Uranus  
Neptune
```

MutableList: add, update, remove

KOTLIN

```
val solarSystem = mutableListOf("Mercury", "Venus", "Earth", "Mars",  
    "Jupiter", "Saturn", "Uranus", "Neptune")  
  
solarSystem.add("Pluto")           // add at the end  
solarSystem.add(3, "Theia")        // insert at index 3  
solarSystem[3] = "Future Moon"     // update index 3  
solarSystem.removeAt(9)            // remove by index: Pluto  
solarSystem.remove("Future Moon")  // remove by value  
  
println(solarSystem.contains("Pluto")) // false  
println("Future Moon" in solarSystem)  // false
```

Two versions of add()

One argument: added at the end. An index first: inserted at that index.

The in operator

A shorter way to write contains().

Only for MutableList

A list created with listOf() cannot add, remove, or update.

Set: unordered, no duplicates

HASH CODES

```
"Kotlin".hashCode()    // -204170231
"Kotlin!".hashCode()   // 1131585312
```

How is this possible?

Every Kotlin object has a hash code: an `Int` from `hashCode()`. A small change to the object gives a very different value. The set uses it to find where an element belongs.

No order, no duplicates

Elements have no position, and the same value can appear only once.

Fast search

Checking if an element is in a set takes about the same time whether it is the first or the hundred thousandth.

More memory

Sets tend to use more memory than lists for the same amount of data.

Use a MutableSet

KOTLIN

```
val solarSystem = mutableSetOf("Mercury", "Venus", "Earth", "Mars",  
    "Jupiter", "Saturn", "Uranus", "Neptune")  
  
println(solarSystem.size)           // 8  
solarSystem.add("Pluto")  
println(solarSystem.size)           // 9  
println(solarSystem.contains("Pluto")) // true  
  
solarSystem.add("Pluto")             // already there: ignored  
println(solarSystem.size)           // 9  
  
solarSystem.remove("Pluto")  
println(solarSystem.size)           // 8  
println(solarSystem.contains("Pluto")) // false
```

No index

add() and remove() take only the element. There is no position.

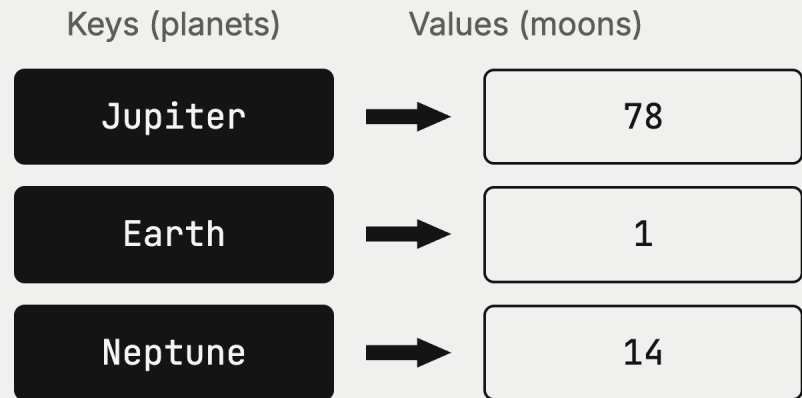
Duplicates are ignored

Adding Pluto a second time leaves the size at 9.

in works here too

"Pluto" in solarSystem is the same as contains().

Map: keys mapped to values



A key and its value together are called a key-value pair.

Keys mapped to values

It is called a map because each unique key is mapped to a value.

Keys are unique, values are not

Two keys can map to the same value: Mercury and Venus both have 0 moons.

Fast lookup by key

Generally faster than searching through a large list with `indexOf()`.

Create a map with mapOf()

WITH EXPLICIT TYPES

```
mutableMapOf< key type , value type >()
```

WITH INITIAL VALUES

```
val map name = mapOf(  
    key to value ,  
    key to value  
)
```

Two generic types: one for keys, one for values. With initial values, Kotlin infers them.

KOTLIN

```
val solarSystem = mutableMapOf(  
    "Mercury" to 0,  
    "Venus" to 0,  
    "Earth" to 1,  
    "Mars" to 2,  
    "Jupiter" to 79,  
    "Saturn" to 82,  
    "Uranus" to 27,  
    "Neptune" to 14  
)
```

Read, add, update, and remove

KOTLIN

```
println(solarSystem.size)           // 8

solarSystem["Pluto"] = 5             // add a new pair
println(solarSystem.size)           // 9
println(solarSystem["Pluto"])        // 5
println(solarSystem.get("Theia"))    // null: no such key

solarSystem.remove("Pluto")
println(solarSystem.size)           // 8

solarSystem["Jupiter"] = 78         // update an existing key
println(solarSystem["Jupiter"])     // 78
```

One syntax, three jobs

Subscript syntax reads a value, adds a new pair, or updates an existing key.

A missing key gives null

[] and get() return null when the key is not in the map.

remove() takes the key

It returns the removed value, or null if the key was not there.

Choosing the right collection

Collection	Ordered	Duplicates	Size	Use it when
Array	Yes, by index	Allowed	Fixed	The number of values never changes
List	Yes, by index	Allowed	Resizable	You need an ordered sequence that can grow
Set	No	Not allowed	Resizable	Values must be unique, or you search often
Map	By key	Keys are unique	Resizable	You look up a value by its key

Scrolling lists in Compose are built on **List**. That is what this week's lab uses.

Summary

01

Collections store multiple values, typically of the same data type.

02

Arrays store ordered data of the same type, and have a fixed size.

03

Lists are a resizable, ordered collection.

04

Sets are unordered collections and cannot contain duplicates.

05

Maps store pairs of unique keys and values.

06

listOf, setOf, mapOf are read-only. The mutable versions can be changed.

What does this print?

```
val planets = arrayOf("Mercury", "Venus", "Earth", "Mars")  
println(planets[4])
```

A Mars

B Earth

C null

D It throws an exception

Exception: the last index is 3

```
val planets = arrayOf("Mercury", "Venus", "Earth", "Mars")
println(planets[4])
// ArrayIndexOutOfBoundsException
```

Why

Arrays are zero-indexed. Four elements means indexes 0, 1, 2 and 3. There is no index 4.

The fix

Mars is the fourth element, so it is at **planets[3]**.

Does this compile?

```
val planets = listOf("Mercury", "Venus", "Earth")  
planets.add("Mars")
```

A Yes

B No: add() needs an index

C No: the list is read-only

No: listOf() is read-only

```
val planets = listOf("Mercury", "Venus", "Earth")
planets.add("Mars")
// error: unresolved reference add
```

Why

listOf() returns a List, which is read-only. The add() method only exists on MutableList.

The fix

Create the list with **mutableListOf()** instead.

What does this print?

```
val planets = mutableSetOf("Earth", "Mars")  
planets.add("Mars")  
planets.add("Venus")  
println(planets.size)
```

A 2

B 3

C 4

D Compile error

3: the duplicate is ignored

```
val planets = mutableSetOf("Earth", "Mars")
planets.add("Mars")           // already in the set
planets.add("Venus")          // added
println(planets.size)         // 3
```

Why

A set cannot contain duplicates. Mars is already there, so the first add() changes nothing.

Compare with a list

With **mutableListOf()** the same code would print 4, because lists allow duplicates.

What does this print?

```
val moons = mutableMapOf("Earth" to 1, "Mars" to 2)
moons["Mars"] = 3
println(moons.size)
println(moons["Venus"])
```

A 2, then null

B 3, then null

C 2, then 0

D 3, then an exception

2, then null

```
val moons = mutableMapOf("Earth" to 1, "Mars" to 2)
moons["Mars"] = 3           // updates Mars, no new pair
println(moons.size)         // 2
println(moons["Venus"])     // null
```

Why 2

Mars is already a key, so assigning to it updates the value. No pair is added and the size stays 2.

Why null

Venus is not a key. A map returns null for a missing key instead of throwing an exception.

Higher-order functions with collections

Sorting, filtering and more in a single line of code. We format a bakery's cookie menu in different ways.

Visit

`forEach`

Transform

`map / filter / sortBy`

Combine

`groupBy / fold`

What is a higher-order function?

FROM LAST WEEK

```
repeat(3) {  
    println("Hello")  
}
```

TODAY, ON COLLECTIONS

```
forEach · map · filter  
groupBy · fold · sortBy
```

A function that takes a function

A higher-order function takes another function as a parameter, and/or returns one. You already used one: `repeat()`.

The function you pass is a lambda

With trailing lambda syntax, it goes in curly braces right after the call.

Why they matter for collections

Common tasks like sorting or filtering take much less code.

Starter code: the cookie menu

KOTLIN

```
class Cookie(  
    val name: String,  
    val softBaked: Boolean,  
    val hasFilling: Boolean,  
    val price: Double  
)  
  
val cookies = listOf(  
    Cookie(...),    // 7 cookies  
    ...  
)
```

name	softBaked	hasFilling	price
Chocolate Chip	false	false	1.69
Banana Walnut	true	false	1.49
Vanilla Creme	false	true	1.59
Chocolate Peanut Butter	false	true	1.49
Snickerdoodle	true	false	1.39
Blueberry Tart	true	true	1.79
Sugar and Sprinkles	false	false	1.39

forEach(): visit every item

```
forEach(action: (T) → Unit)
```

Runs the function you pass once for each item in the collection, like a for loop.

T is the type of the items. The lambda has a single parameter, so you can leave out its name and call it **it**.

KOTLIN

```
cookies.forEach {  
    println("Menu item: $it")  
}
```

Output:

```
Menu item: Cookie@5a10411  
Menu item: Cookie@68de145  
Menu item: Cookie@27fa135a  
...
```

It runs, but each line shows only the type name and an identifier. Cookie is a regular class, not a **data class**.

Expressions in strings: \${ }

```
println("Menu item: $it")
```

```
Output: Menu item: Cookie@5a10411
```

```
println("Menu item: $it.name")
```

```
Output: Menu item: Cookie@5a10411.name
```

```
println("Menu item: ${it.name}")
```

```
Output: Menu item: Chocolate Chip
```

\$ plus a variable name

Inserts the variable itself. For a Cookie that is its type and an identifier.

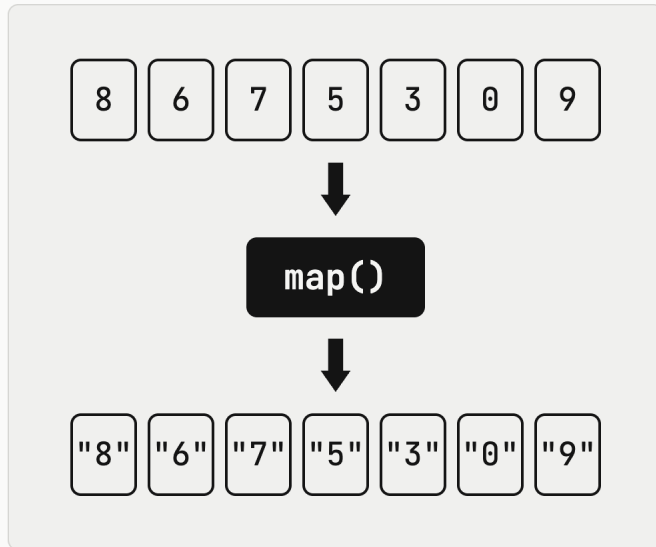
\$it.name does not work

Only it is inserted. The text .name is printed as it is.

\${ } takes an expression

Access a property, do math, call a function. The result is inserted into the string.

map(): transform every item



KOTLIN

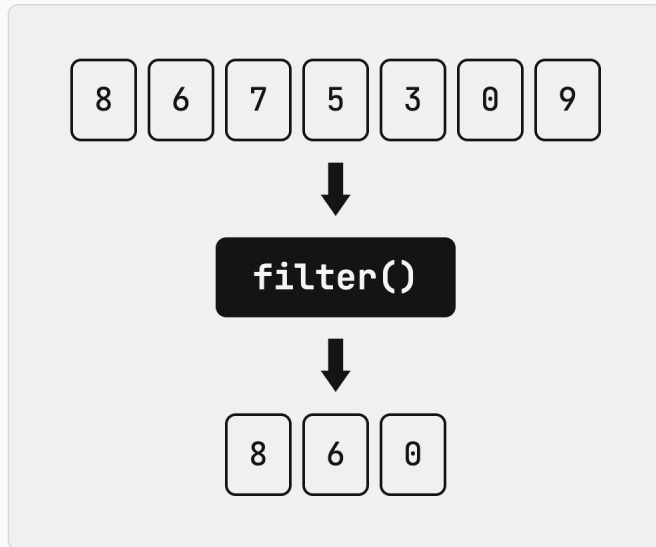
```
val fullMenu = cookies.map {  
    "${it.name} - ${it.price}"  
}  
  
println("Full menu:")  
fullMenu.forEach {  
    println(it)  
}
```

OUTPUT

```
Full menu:  
Chocolate Chip - $1.69  
Banana Walnut - $1.49  
Vanilla Creme - $1.59  
Chocolate Peanut Butter - $1.49  
Snickerdoodle - $1.39  
Blueberry Tart - $1.79  
Sugar and Sprinkles - $1.39
```

`map()` returns a new collection with the **same number of elements**, often of another type: `List<Cookie>` becomes `List<String>`. The first `$` in `${it.price}` is a plain dollar sign.

filter(): keep what passes a test



KOTLIN

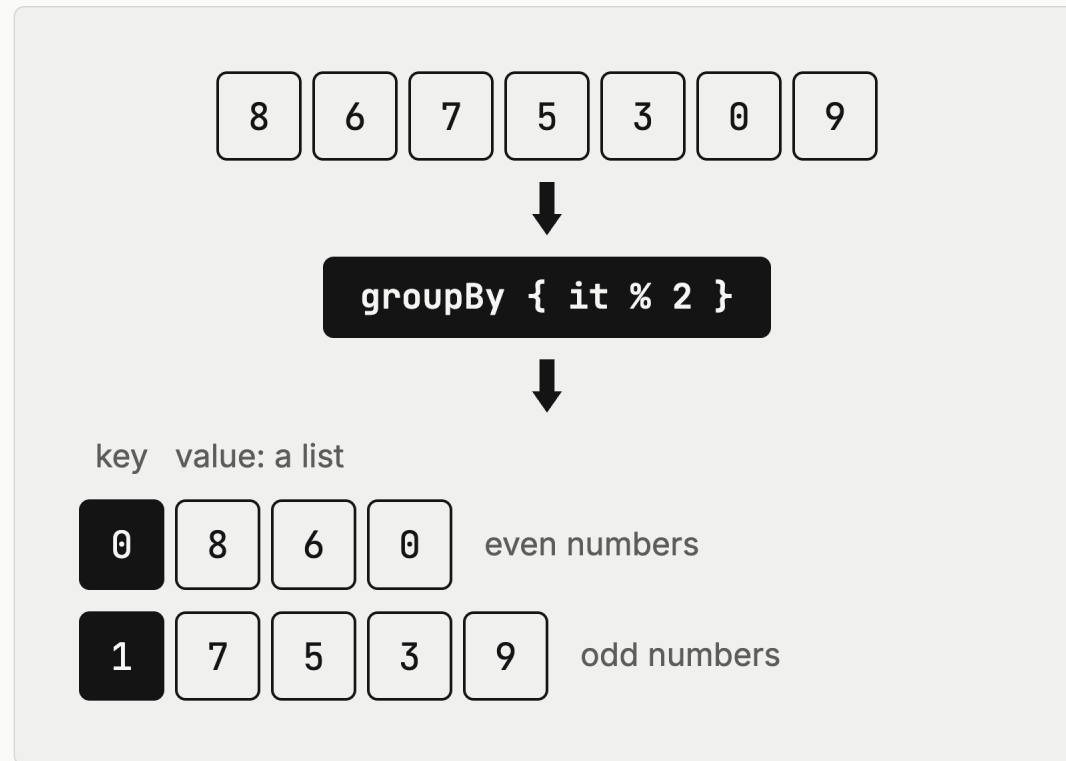
```
val softBakedMenu = cookies.filter {  
    it.softBaked  
}  
  
println("Soft cookies:")  
softBakedMenu.forEach {  
    println("${it.name} - ${it.price}")  
}
```

OUTPUT

```
Soft cookies:  
Banana Walnut - $1.49  
Snickerdoodle - $1.39  
Blueberry Tart - $1.79
```

The lambda returns a Boolean: **true** keeps the item, **false** leaves it out. The result has the same type, List<Cookie>, and is the same size or smaller.

groupBy(): turn a list into a map



A list becomes a map

Each unique value returned by the lambda becomes a key in the resulting map.

The values are lists

For each key: all the items that produced that return value.

The modulo operator %

Returns the remainder of a division. `it % 2` is 0 for even numbers and 1 for odd numbers.

Group the menu by softBaked

KOTLIN

```
val groupedMenu = cookies.groupBy { it.softBaked }

val softBakedMenu = groupedMenu[true] ?: listOf()
val crunchyMenu = groupedMenu[false] ?: listOf()

println("Soft cookies:")
softBakedMenu.forEach {
    println("${it.name} - ${it.price}")
}
println("Crunchy cookies:")
crunchyMenu.forEach {
    println("${it.name} - ${it.price}")
}
```

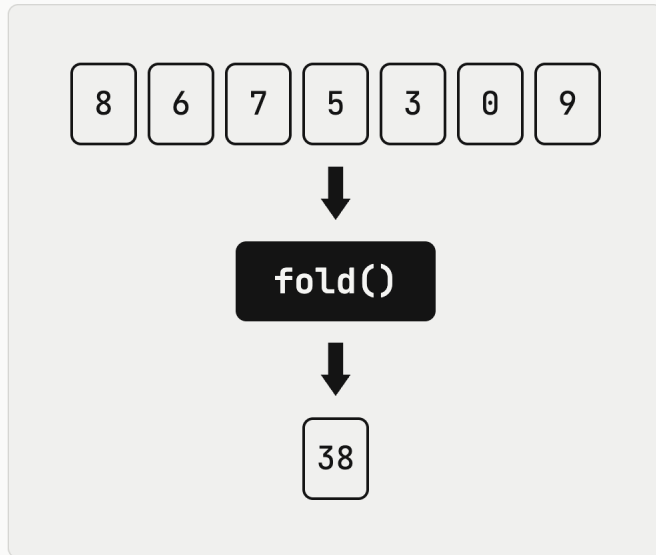
OUTPUT

```
Soft cookies:
Banana Walnut - $1.49
Snickerdoodle - $1.39
Blueberry Tart - $1.79
Crunchy cookies:
Chocolate Chip - $1.69
Vanilla Creme - $1.59
Chocolate Peanut Butter - $1.49
Sugar and Sprinkles - $1.39
```

Result: Map<Boolean, List<Cookie>>

Reading from a map can give null, so the Elvis operator `?:` supplies an empty list instead.

fold(): one value from a collection



KOTLIN

```
val totalPrice = cookies.fold(0.0) { total, cookie →  
    total + cookie.price  
}
```

```
println("Total price: ${totalPrice}")
```

Output: Total price: \$10.83

`fold()` takes an **initial value** and a lambda with two parameters. Whatever the lambda returns becomes **total** for the next item.

fold() step by step

cookie	total	+ price	returns
Chocolate Chip	0.00	1.69	1.69
Banana Walnut	1.69	1.49	3.18
Vanilla Creme	3.18	1.59	4.77
Chocolate Peanut Butter	4.77	1.49	6.26
Snickerdoodle	6.26	1.39	7.65
Blueberry Tart	7.65	1.79	9.44
Sugar and Sprinkles	9.44	1.39	10.83

Running totals are shown rounded to two decimals.

Initial value

The starting point: 0.0 here. Its type decides the type of the result.

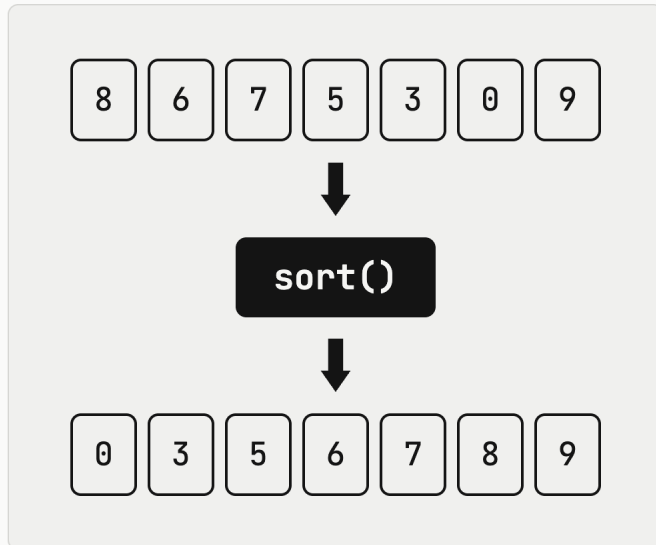
Accumulator: total

A running total. It holds what the lambda returned the previous time.

Element: cookie

The current item of the collection, one per call.

sortedBy(): sort by a property



KOTLIN

```
val alphabeticalMenu = cookies.sortedBy {  
    it.name  
}  
  
println("Alphabetical menu:")  
alphabeticalMenu.forEach {  
    println(it.name)  
}
```

OUTPUT

```
Alphabetical menu:  
Banana Walnut  
Blueberry Tart  
Chocolate Chip  
Chocolate Peanut Butter  
Snickerdoodle  
Sugar and Sprinkles  
Vanilla Creme
```

A list of Cookie objects has no natural order: Kotlin can't know which property you mean. The lambda returns the property to sort by. The result is still a `List<Cookie>`.

Choosing the right function

Function	The lambda returns	You get back	Cookie example
forEach	Nothing	Nothing: it just runs the code	Print every cookie
map	A new value for the item	A collection of the same size	Names with prices
filter	true or false	The same size or smaller	Soft-baked cookies only
groupBy	A key for the item	A map of key to list	Soft and crunchy menus
fold	The new running total	A single value	Total price
sortedBy	The value to sort by	A sorted collection	Alphabetical menu

Summary

01

`forEach()` loops over each element in a collection.

02

Expressions can be inserted into strings with `${ }`.

03

`map()` formats the items, often as a collection of another data type.

04

`filter()` generates a subset of a collection.

05

`groupBy()` splits a collection based on a function's return value.

06

`fold()` turns a collection into a single value. `sortBy()` sorts by a property.

What does this print?

```
val cookie = Cookie("Snickerdoodle", true, false, 1.39)
println("Name: $cookie.name")
```

A Name: Snickerdoodle

B Something like Name:
Cookie@5a10411.name

C Name: \$cookie.name

D Compile error

The object, then the text `.name`

```
val cookie = Cookie("Snickerdoodle", true, false, 1.39)
println("Name: $cookie.name") // Name: Cookie@5a10411.name
```

Why

Without curly braces only **\$cookie** is inserted. The rest, `.name`, is ordinary text.

The fix

Write **`${cookie.name}`** to insert the value of the property.

What does this print?

```
val numbers = listOf(8, 6, 7, 5, 3, 0, 9)
val result = numbers.map { it % 2 == 0 }
println(result.size)
```

A 3

B 4

C 7

D 2

7: map() keeps every element

```
val numbers = listOf(8, 6, 7, 5, 3, 0, 9)
val result = numbers.map { it % 2 == 0 }
// [true, true, false, false, false, true, false]
println(result.size)      // 7
```

Why

map() transforms each item and always returns the same number of elements. Here each number becomes true or false.

Compare with filter

numbers.filter { it % 2 == 0 } would keep only 8, 6 and 0, so its size would be 3.

What does this print?

```
val words = listOf("apple", "avocado", "banana", "cherry")
val groups = words.groupBy { it.first() }
println(groups.size)
```

A 4

B 3

C 2

D 1

ANSWER: B · 3 OF 4

3: one key per first letter

```
val words = listOf("apple", "avocado", "banana", "cherry")
val groups = words.groupBy { it.first() }
// {a=[apple, avocado], b=[banana], c=[cherry]}
println(groups.size)      // 3
```

Why

Each unique value returned by the lambda becomes a key. The first letters are a, a, b and c: three different keys.

What size counts

The size of a map is its number of key-value pairs, not the number of words.

What does this print?

```
val numbers = listOf(1, 2, 3, 4)
val result = numbers.fold(10) { total, n → total + n }
println(result)
```

A 10

B 20

C 24

D 0

ANSWER: B · 4 OF 4

20: the total starts at 10

```
val numbers = listOf(1, 2, 3, 4)
val result = numbers.fold(10) { total, n → total + n }
// 10 + 1 + 2 + 3 + 4
println(result)      // 20
```

Why

`fold()` starts from the initial value, 10, and then adds each element to the running total.

Compare

With **`fold(0)`** the result would be 10, the plain sum of the list.

Keep practicing

Put today into practice, at your own pace.

Next

5. Practice: Classes and Collections

Activity 5 of the pathway. A set of tasks to complete in the Kotlin Playground.

Optional

Take the pathway quiz

On the course website, to test yourself and earn the More Kotlin fundamentals badge.

This week

Lab 4: Scrollable lists

It uses a data class, a List, and a lambda. Complete the prerequisite before the lab.